PROGRAMMING & CS FUNDAMENTALS

OOP Principles Visual Guide

Encapsulation, inheritance, polymorphism — and knowing when NOT to use them.

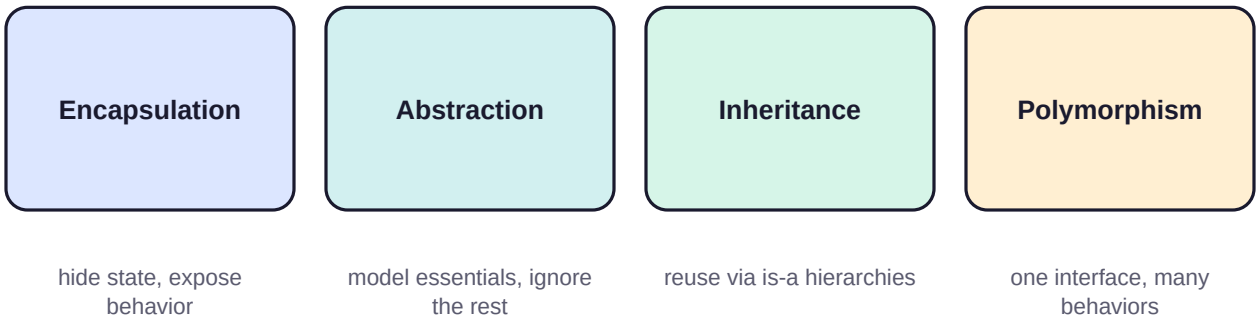
ANIMATED WALKTHROUGH

Step through pages to play the animation

STEP 1/6

The Four Pillars

One idea each — the rest of OOP is commentary.



- OOP bundles DATA with the CODE that operates on it — an object owns its state.
- Class = blueprint; object = instance; method = function bound to an object.
- The pillars reinforce each other: encapsulation enables safe polymorphism.

STEP 2/6

Encapsulation: Hide the Gears

Private state, public behavior — invariants enforced at the boundary.

```
class BankAccount:
    def __init__(self): self.__balance = 0   # private by convention
    def deposit(self, amt):
        if amt <= 0: raise ValueError        # invariant HERE
        self.__balance += amt
    def balance(self): return self.__balance  # read-only view
```

balance = -500?

impossible: no setter

deposit(100)

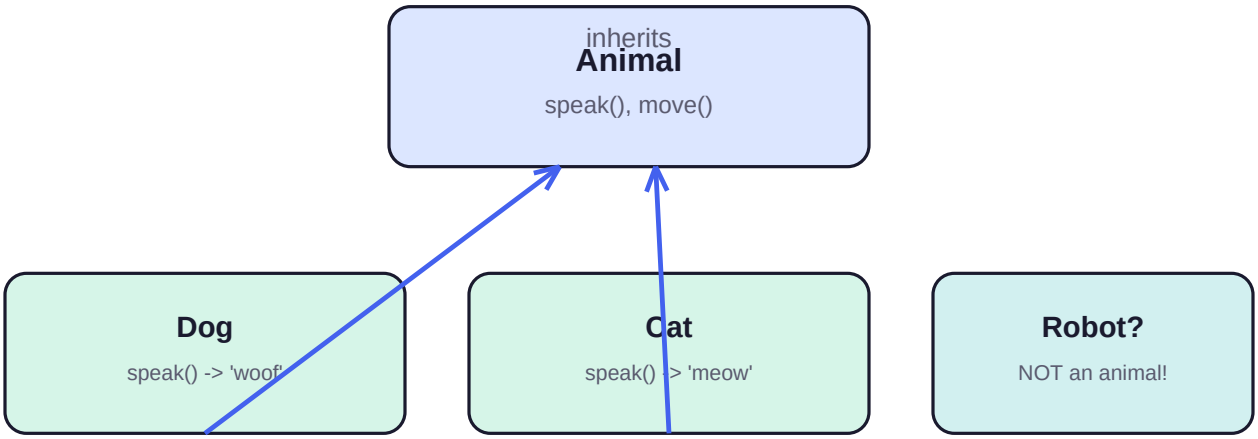
validated at the gate

- If every method preserves the invariant, the object can never be invalid.
- Exposing raw fields invites every caller to break your assumptions — don't.
- Tell, don't ask: call account.withdraw(x), don't read balance and do math outside.

STEP 3/6

Inheritance: The Is-A Chain

Subclasses reuse and specialize — power with a coupling cost.




```
class Dog(Animal):
    def speak(self): return 'woof'   # override
# Dog reuses Animal.move() for free — but is now COUPLED to Animal
```

Fragile base class: changing Animal can silently break every subclass. Inheritance is the strongest coupling in OOP — spend it wisely.

STEP 4/6

Polymorphism: One Call, Many Behaviors

Code to the interface — the object decides what 'speak' means.

```
def chorus(animals):
    for a in animals: print(a.speak())  # no ifs, no type checks

chorus([Dog(), Cat(), Bird()])
# woof / meow / chirp — dynamic dispatch picks the method
```

a.speak()

compile-time: unknown | run-time: resolved by actual type

- Subtype polymorphism: subclasses stand in for the base type (Liskov's rule).
- Duck typing: 'if it speaks like a duck' — Python/JavaScript skip the hierarchy entirely.
- Kills switch-on-type chains: new animal = new class, zero edits to chorus().

STEP 5/6

Composition Over Inheritance

Has-a beats is-a: assemble behavior instead of inheriting it.

Brittle: deep hierarchy

Vehicle -> Car -> SportsCar -> ConvertibleSportsCar ...

Flexible: compose parts

```
class Car:
    def __init__(self, engine, brakes):  # has-a engine, has-a brakes
        self.engine, self.brakes = engine, brakes
# swap ElectricEngine for V8 — no hierarchy surgery
```

- Inheritance locks behavior at COMPILE time; composition swaps it at RUN time.
- Strategy pattern IS composition: the algorithm becomes a swappable part.
- Rule of thumb: inherit for true is-a (rare); compose for everything else.

The Gang of Four said it first: 'favor object composition over class inheritance.' Thirty years later, still the best OOP advice.

STEP 6/6

OOP Traps

Where object thinking goes wrong.

- God objects: one class doing everything — encapsulation without boundaries.
- Inheritance for code reuse alone: reuse is composition's job; inheritance is for polymorphism.
- Getters/setters for every field: that's a struct with extra steps, not encapsulation.
- Premature hierarchies: three levels deep before the second use case exists.
- Anemic domain models: data classes + procedural services — OOP syntax, procedural soul.
- Not everything is an object: pure transforms and pipelines often read better as functions.

STEP ✓/✓

Key Takeaways

OOP Principles — Remember

- Encapsulation: private state + validating methods keep objects always-valid.
- Inheritance reuses via is-a but couples tightly — fragile base classes bite.
- Polymorphism: code to the interface; dynamic dispatch picks the behavior.
- Prefer composition (has-a, swappable at run time) over inheritance (locked at compile time).
- Getters-for-everything and god objects are OOP-shaped procedural code.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev